

INFRAESTRUCTURA COMO CODIGO

Ander Vera Gorozabel

Contenido

1. INTRODUCCIÓN	2
2. OBJETIVOS	2
3. FASES DEL PROYECTO	2
4. ANÁLISIS.....	2
5. DISEÑO.....	3
6. IMPLEMENTACIÓN Y CONFIGURACIÓN.....	4
6.1. Preparación del Entorno	4
6.2. Inicialización de la Infraestructura.....	4
6.3. Despliegue del Servidor	5
6.4. Gestión de Configuración con Ansible	5
6.4.1. Creación de la lógica de automatización (Playbooks).....	6
6.4.2. Ejecución y verificación del despliegue.....	6
6.4.3. Implementación de capas de seguridad (Hardening)	7
6.4.4. Análisis técnico de la política de seguridad	8

1. INTRODUCCIÓN

En el entorno actual de la administración de sistemas, la configuración manual de servidores es un proceso propenso a errores e ineficiente. Este proyecto nace de la necesidad de automatizar el ciclo de vida de la infraestructura informática. Mediante la **Infraestructura como Código (IaC)**, trataremos la creación y configuración de servidores como si fuera el desarrollo de una aplicación, permitiendo despliegues rápidos, seguros y documentados automáticamente.

2. OBJETIVOS

El objetivo final es disponer de un sistema que permita levantar un servidor operativo y seguro con un solo comando.

- **Automatización de la Infraestructura:** Lograr que, mediante instrucciones escritas, aparezcan máquinas virtuales listas para usarse sin intervención manual.
- **Despliegue Automático de Servicios:** Instalar y configurar el software necesario (como el servidor web Apache) nada más encenderse la máquina.
- **Securización:** Aplicar una capa de seguridad automática que incluya firewalls y protección contra intrusos mediante código.

3. FASES DEL PROYECTO

El proyecto se divide en las siguientes etapas cronológicas:

1. **Fase de Laboratorio:** Instalación de VirtualBox y Vagrant.
2. **Fase de Aprovisionamiento:** Creación del archivo Vagrantfile y primer despliegue de Ubuntu.
3. **Fase de Automatización:** Creación de Playbooks de Ansible para instalar Apache.
4. **Fase de Hardening (Seguridad):** Automatización de UFW y Fail2Ban.
5. **Fase de Pruebas y Documentación:** Verificación final y redacción de la memoria.

4. ANÁLISIS

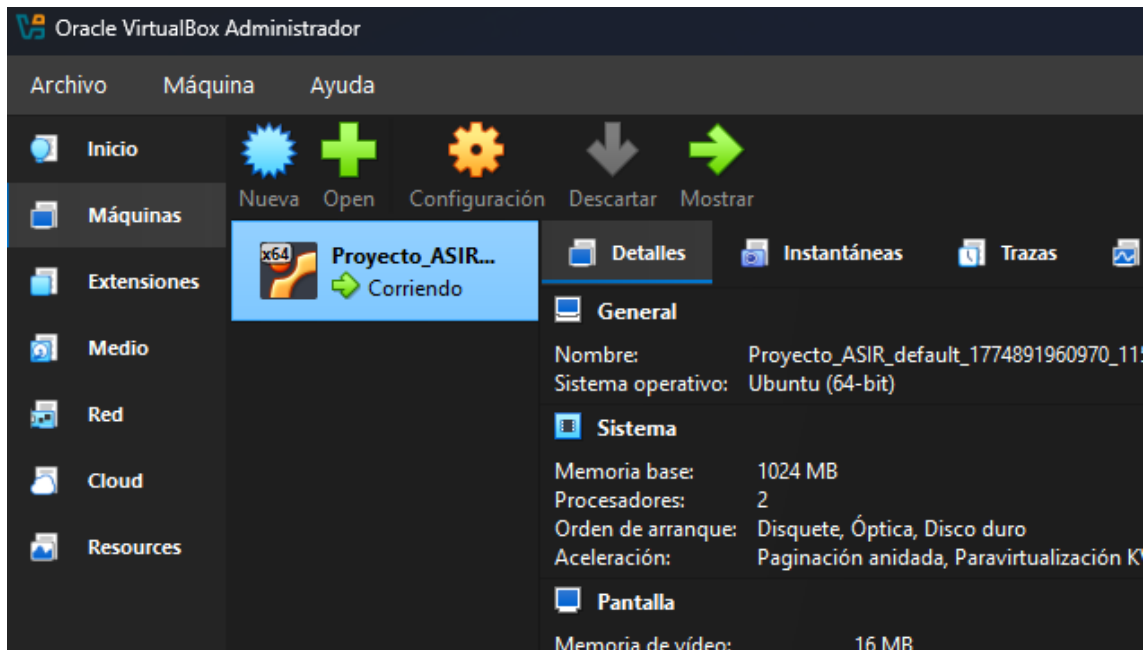
Se han seleccionado las siguientes herramientas por su viabilidad y peso en el sector actual:

- **Vagrant + VirtualBox:** VirtualBox actúa como hipervisor gratuito y Vagrant como el software encargado de crear las máquinas virtuales.
- **Ansible:** Herramienta basada en Python que utiliza archivos YAML (Playbooks) para dar instrucciones paso a paso a los servidores.
- **Ubuntu 22.04 LTS:** Sistema operativo Linux base para garantizar estabilidad y seguridad.

5. DISEÑO

El diseño se basa en una arquitectura de Nodo de Control (PC Host) y Nodo Gestionado (Servidor Ubuntu).

- **Componentes Hardware:** Procesador con virtualización habilitada y memoria RAM suficiente para soportar las instancias virtuales.
- **Esquema Funcional:** Vagrant lee el archivo de configuración y se comunica con la API de VirtualBox para asignar recursos de red y disco.



6. IMPLEMENTACIÓN Y CONFIGURACIÓN

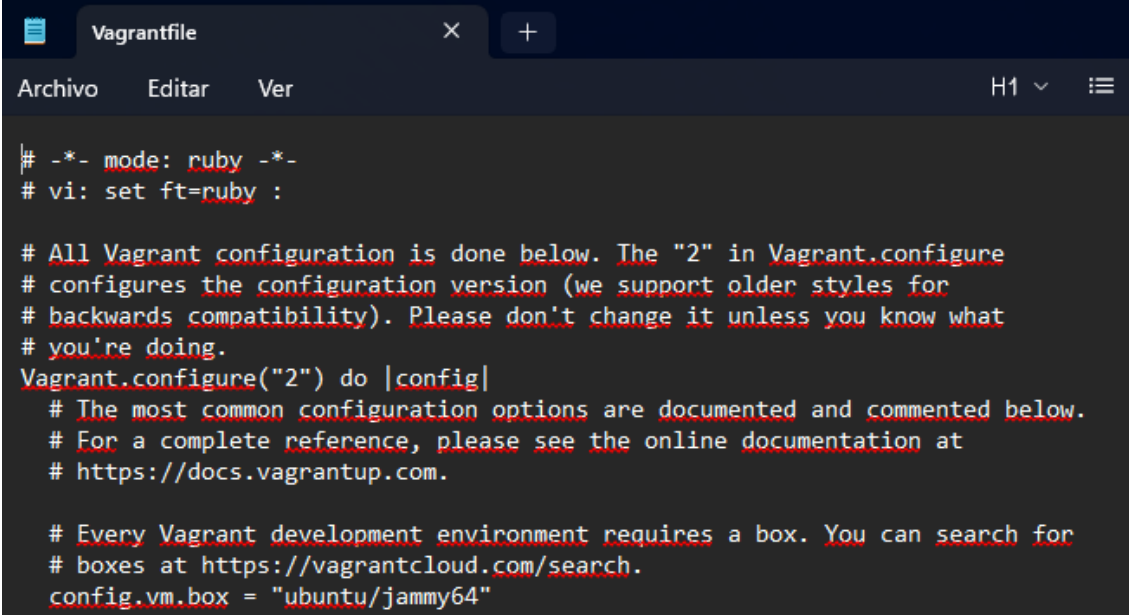
6.1. Preparación del Entorno

Se ha procedido a la instalación de VirtualBox y Vagrant. Tras la instalación, se verificó el estado de la virtualización en la BIOS del sistema.

6.2. Inicialización de la Infraestructura

Mediante la terminal, se ejecutó el comando **vagrant init ubuntu/jammy64** para inicializar el entorno, el cual genera un archivo Vagrantfile:

```
PS C:\Users\Ander\Desktop\Proyecto_ASIR> vagrant init ubuntu/jammy64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS C:\Users\Ander\Desktop\Proyecto_ASIR> |
```



The screenshot shows a code editor window titled 'Vagrantfile'. The editor contains the following text:

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

# All Vagrant configuration is done below. The "2" in Vagrant.configure
# configures the configuration version (we support older styles for
# backwards compatibility). Please don't change it unless you know what
# you're doing.
Vagrant.configure("2") do |config|
  # The most common configuration options are documented and commented below.
  # For a complete reference, please see the online documentation at
  # https://docs.vagrantup.com.

  # Every Vagrant development environment requires a box. You can search for
  # boxes at https://vagrantcloud.com/search.
  config.vm.box = "ubuntu/jammy64"
```

6.3. Despliegue del Servidor

Al ejecutar **vagrant up**, el sistema descarga la imagen oficial y configura el servidor automáticamente.

```
PS C:\Users\Ander\Desktop\Proyecto_ASIR> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Setting the name of the VM: Proyecto_ASIR_default_1774891960970_11551
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
```

6.4. Gestión de Configuración con Ansible

Una vez desplegada la infraestructura base con Vagrant, se ha procedido a la instalación de Ansible como motor de automatización.

```
vagrant@ubuntu-jammy:~$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  cowsay sshpass
The following NEW packages will be installed:
  ansible
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 17.5 MB of archives.
After this operation, 204 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 ansible all 2.10.7+merged+base+2.10.8+dfsg-1 [17.5 MB]
Fetched 17.5 MB in 1s (14.4 MB/s)
Selecting previously unselected package ansible.
(Reading database ... 68053 files and directories currently installed.)
Preparing to unpack .../ansible_2.10.7+merged+base+2.10.8+dfsg-1_all.deb ...
Unpacking ansible (2.10.7+merged+base+2.10.8+dfsg-1) ...
Progress: [ 20%] [#####.....]
```

6.4.1. Creación de la lógica de automatización (Playbooks)

Una vez preparado el motor de Ansible, se procede a la creación de los "Playbooks". Estos archivos representan la infraestructura como datos, permitiendo que cualquier administrador pueda leer y entender la configuración del servidor sin necesidad de manuales externos. Para el despliegue del servicio, se ha creado el archivo `instalar_apache.yml`

```
GNU nano 6.2                                instalar_apache.yml
- name: Configuración automatica de servidor web
  hosts: localhost
  connection: local
  become: yes
  tasks:
    - name: Instalar paquete| apache2
      apt:
        name: apache2
        state: present
        update_cache: yes

    - name: Asegurar que el servicio Apache este activo
      service:
        name: apache2
        state: started
        enabled: yes
```

6.4.2. Ejecución y verificación del despliegue

Para aplicar los cambios definidos en el código, se hace uso del comando **ansible-playbook**. Durante este proceso, Ansible analiza el estado actual del servidor y, si detecta que el servicio no está presente, procede a su configuración automática.

```
vagrant@ubuntu-jammy:~/mi-proyecto$ ansible-playbook instalar_apache.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Configuración automatica de servidor web] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Instalar paquete apache2] *****
changed: [localhost]

TASK [Asegurar que el servicio Apache este activo] *****
ok: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

vagrant@ubuntu-jammy:~/mi-proyecto$ |
```

6.4.3. Implementación de capas de seguridad (Hardening)

Una vez asegurada la disponibilidad del servicio web, el siguiente hito crítico es la protección del nodo. Siguiendo los principios de DevSecOps, se ha diseñado una política de seguridad que se aplica de forma automática al servidor nada más ser desplegado.

```
GNU nano 6.2                                seguridad.yml *
```

```
- name: Securizacion automatica del servidor
  hosts: localhost
  connection: local
  become: yes
  tasks:
    - name: Instalar Firewall (UFW) y Fail2Ban
      apt:
        name: ["ufw", "fail2ban"]
        state: present

    - name: Permitir trafico Web (puerto 80)
      ufw:
        rule: allow
        port: '80'
        proto: tcp

    - name: Permitir trafico SSH (puerto 22)
      ufw:
        rule: allow
        port: '22'
        proto: tcp

    - name: Activar el Firewall y establecer politica por defecto
      ufw:
        state: enabled
        policy: deny|
```


6.4.4. Análisis técnico de la política de seguridad

Gestión de puertos: Se abren específicamente los puertos 80 (HTTP) para el servicio web y 22 (SSH) para el mantenimiento remoto.

Política de denegación: El comando final establece una política de "Deny" por defecto. Esto significa que cualquier conexión que no haya sido autorizada explícitamente será bloqueada automáticamente, siguiendo el principio de mínimo privilegio.

```
vagrant@ubuntu-jammy:~/mi-proyecto$ ansible-playbook seguridad.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Securizacion automatica del servidor] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Instalar Firewall (UFW) y Fail2Ban] *****
changed: [localhost]

TASK [Permitir trafico Web (puerto 80)] *****
changed: [localhost]

TASK [Permitir trafico SSH (puerto 22)] *****
changed: [localhost]

TASK [Activar el Firewall y establecer politica por defecto] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

vagrant@ubuntu-jammy:~/mi-proyecto$ |
```

```
vagrant@ubuntu-jammy:~/mi-proyecto$ sudo ufw status
Status: active

To Action From
--
80/tcp ALLOW Anywhere
22/tcp ALLOW Anywhere
80/tcp (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)

vagrant@ubuntu-jammy:~/mi-proyecto$ |
```